

10Pearls AQI Predictor

Project Report

A 24h / 48h / 72h Air Quality Index forecasting system for Karachi, Lahore, and Islamabad, built on an automated, serverless-style ML pipeline.

Data source: Open-Meteo Weather & Air Quality APIs
Storage: Backblaze B2 (feature store + model registry)
Automation: GitHub Actions | Dashboard: Streamlit

Live app: <https://10pearls-aqi-predictor-by-muhammad-sahban.streamlit.app/>

1. Project Overview

This project forecasts US AQI 24, 48, and 72 hours ahead for three Pakistani cities: Karachi, Lahore, and Islamabad. The system is fully automated end to end — hourly data collection, daily retraining — and serves live predictions through a Streamlit dashboard. The architecture uses no self-managed servers: GitHub Actions provides scheduled compute, Backblaze B2 provides the feature store and model registry, and Streamlit Community Cloud hosts the live dashboard.

2. Data

Attribute	Value
Source	Open-Meteo Weather API + Open-Meteo Air Quality API
Backfill range	2025-08-01 to 2026-08-01 (1 year)
Total rows	~24,000 hourly rows across 3 cities combined
Cities	Karachi, Lahore, Islamabad

3. Feature Engineering

a) AQI lag / rolling features

Computed using only values from **before** the current timestamp (AQI 1h/3h/24h/72h ago, rolling mean/std over 6h/24h/72h windows, hour-over-hour change rate). These never use same-timestamp or future values, so they cannot leak information from the target into training.

b) Weather features — used twice per row

Weather variables (temperature, humidity, pressure, wind, etc.) are used both as the **current** weather at time T , and again as the weather **at the target time** ($T + \text{horizon}$), reconstructed by shifting historical weather forward. This second copy stands in for what a live forecast would have said, since real inference uses Open-Meteo's actual forecast for that same future timestamp — keeping training and live serving structurally consistent.

c) Time features

Hour, day of week, and month — computed for both the current and the target timestamp, since AQI follows daily and seasonal cycles independent of any lag value.

4. Models Compared, and Why These Three

Three candidate models were trained and compared per horizon, and the best one (by test-set RMSE) was kept automatically:

- **Ridge Regression** — an interpretable, extremely fast, extremely small (kilobytes, not megabytes) linear baseline. Its regularization also handles the multicollinearity that's unavoidable here, since many lag/rolling AQI features are naturally correlated with each other.
- **Random Forest** — a bagged ensemble of decision trees. Captures non-linear relationships and feature interactions without requiring feature scaling or heavy tuning, and is naturally robust to outliers.
- **Gradient Boosting** — a sequential boosted-tree ensemble. Included as a second, different non-linear approach: trees are built sequentially, each correcting the previous ones' errors, which on many tabular problems beats bagging.

Together these three span a deliberately wide spread — linear vs. non-linear, and (among the non-linear pair) bagging vs. boosting — giving a real comparison across different modeling assumptions rather than three variations on the same idea. Deep learning was considered but not included as a primary candidate given the dataset size (~24,000 rows) is on the smaller side for that approach to reliably outperform well-tuned tree/linear models.

5. Results — Training Summary

All metrics are computed on a held-out, time-based test split (the model never trains on data from after the test period).

Horizon	Selected Model	RMSE	MAE	R ²
24h	Ridge	17.83	12.51	0.792
48h	Random Forest	21.78	15.45	0.688
72h	Random Forest	24.73	18.01	0.598

Full candidate comparison

Horizon	Model	RMSE	MAE	R ²
24h	Ridge (selected)	17.83	12.51	0.792
24h	Random Forest	18.10	12.42	0.786
24h	Gradient Boosting	21.48	14.11	0.698
48h	Random Forest (selected)	21.78	15.45	0.688
48h	Gradient Boosting	22.26	15.98	0.674
48h	Ridge	23.71	17.17	0.631

Horizon	Model	RMSE	MAE	R ²
72h	Random Forest (selected)	24.73	18.01	0.598
72h	Gradient Boosting	25.23	18.49	0.582
72h	Ridge	25.18	19.12	0.583

Observation: at 24h, Ridge and Random Forest are extremely close (RMSE 17.83 vs 18.10), with Gradient Boosting notably worse. At 48h and 72h, Random Forest pulls ahead more clearly and Ridge falls behind — consistent with AQI's relationship to weather becoming more non-linear at longer horizons, where a purely linear model has less to work with.

6. Persistence Baseline Validation

Before trusting these models, each one was checked against the simplest possible forecast: **“AQI will be whatever it is right now”** (the persistence baseline), evaluated on the exact same test rows as the real model.

Horizon	Baseline RMSE	Model RMSE	Improvement	Verdict
24h	20.96	17.83	14.9%	Beats baseline
48h	28.54	21.78	23.7%	Beats baseline
72h	30.00	24.73	17.6%	Beats baseline

All three horizons meaningfully outperform naive persistence, confirming the models learn genuine signal from weather and time features rather than just re-stating the last known AQI value. Interestingly, the **largest** improvement over baseline is at 48h (23.7%), not 24h — the naive baseline degrades faster than the trained model as the horizon grows, which is exactly the behavior you want: the model's advantage over “doing nothing” grows with horizon, even as its absolute accuracy (RMSE) declines.

7. Feature Importance (SHAP)

SHAP analysis was run per horizon using the algorithm matched to whichever model won that horizon (LinearExplainer for Ridge, TreeExplainer for Random Forest).

Horizon	Top feature	2nd feature	All other 43 features
24h (Ridge)	surface_pressure: 119.34	surface_pressure_target: 80.64	149.03
48h (Random Forest)	us_aqi: 16.66	us_aqi_roll_mean_72h: 3.85	11.92
72h (Random Forest)	us_aqi: 15.65	us_aqi_roll_mean_72h: 6.67	14.50

Notable finding: at 24h, surface pressure (both current and forecasted-for-target-time) is by far the dominant signal — not AQI persistence. This suggests that for next-day prediction specifically, the Ridge model leans heavily on atmospheric pressure patterns rather than simply extrapolating recent pollution levels. At 48h and 72h, current AQI and its 72-hour rolling average become the top two features — the more expected “persistence-driven” pattern for longer-range forecasts — but even there, the combined weight of the other 43 features is close to (48h) or exceeds (72h) the persistence-related features, meaning the model does not rely on persistence alone at any horizon.

8. Architecture and Automation

- **feature_pipeline.py** runs hourly via GitHub Actions, fetching fresh weather/AQI data and writing features to Backblaze B2.
- **training_pipeline.py** runs daily via GitHub Actions, training all 3 candidate models per horizon and keeping only the best.
- **shap_analysis.py** runs immediately after training, same schedule.
- **Prediction/inference happens live**, inside the Streamlit app's own process — not on a schedule, not pre-computed. Every dashboard load (cached ~1 hour) fetches live Open-Meteo data and whichever model is currently newest in B2, so a new day's retrained model is reflected within the hour, with no redeploy needed.
- **Storage policy:** when B2 is configured, models, features, SHAP plots, training summaries, and run logs live only in B2 — nothing is written to local disk or committed to git. Models transfer purely in memory.
- **Live app:** <https://10pearls-aqi-predictor-by-muhammad-sahban.streamlit.app/>

9. Challenges Encountered and Resolutions

- Hopsworks' free-tier cluster provisioning got stuck indefinitely, blocking the originally planned feature store / model registry.
- Vertex AI (the project brief's suggested alternative) required credit card details even for its free tier, which was ruled out.
- **Backblaze B2 was chosen instead:** a genuine cloud object store, free tier with no card required, S3-compatible API.
- Large model uploads to B2 initially failed intermittently with SSL/connection errors during multipart upload — resolved with retry logic (exponential backoff) around every network call, plus the model size reduction below, which made the failure far less likely overall.
- The Random Forest model was initially very large (tens of MB) due to deep, numerous trees (300 estimators, depth 14). Tuning hyperparameters (100 estimators, depth 10, larger minimum leaf size) plus joblib compression reduced model size by roughly 35x with no measurable accuracy loss.
- SHAP's TreeExplainer crashed on the Random Forest model due to boolean-typed one-hot-encoded city columns mixing with float columns. Resolved by casting SHAP input to float64 and switching to SHAP's unified Explainer API, which auto-selects Tree vs. Linear explainer depending on which model won a given horizon.
- GitHub Actions' native scheduled (cron) triggers were not firing reliably — manual runs always worked correctly and quickly, confirming the pipelines themselves were fine, but GitHub's built-in schedule is

best-effort and can be delayed significantly, particularly around the top of every hour when many repositories worldwide are scheduled at once. Resolved by removing the native schedule trigger and using an external free cron service (cron-job.org) to call GitHub's REST API `workflow_dispatch` endpoint on schedule instead, authenticated with a fine-grained personal access token scoped to only this repository.

- Live predictions were initially anchored to the wrong point in time: the code treated “the last row of freshly fetched data” as “now,” but since each fetch pulls ~3 days of history plus ~4 days of Open-Meteo's own forecast, the last row was actually several days in the future — silently shifting both the displayed current AQI and every +24h/+48h/+72h target date forward by days. Resolved by explicitly computing the true current time (in GMT+5, since all three cities share that fixed offset with no daylight saving) and anchoring every prediction to the row matching that real timestamp.
- Streamlit Community Cloud's free tier puts inactive apps to sleep, causing a slow cold-start on the next visit. Resolved with a dedicated GitHub Actions workflow, triggered every 8 hours by the same external cron-job.org scheduler used elsewhere, that simply pings the live app to keep it warm

10. Limitations

- Accuracy degrades with horizon (R^2 drops from 0.792 at 24h to 0.598 at 72h) — expected, since 72h predictions are bounded by how good Open-Meteo's own 72h weather forecast is, not just the model.
- Training uses shifted historical weather as a stand-in for “forecasted weather,” since true historical forecast data isn't available; live inference uses real forecasts. These are structurally identical features but not statistically identical distributions.
- Evaluation uses a time-based, not random, train/test split — correct for time series, but reflects performance on one contiguous recent period rather than an average over many random samples.

11. Conclusion

All three horizons produce models that meaningfully outperform a naive persistence baseline (14.9% to 23.7% RMSE improvement), with different algorithms winning at different horizons (Ridge at 24h, Random Forest at 48h and 72h) — validating the decision to compare multiple model types per horizon rather than assuming one algorithm suits every forecast range. The 24h SHAP result in particular suggests atmospheric pressure carries more next-day predictive signal than recent AQI history, an empirical finding that could inform future feature engineering (e.g. giving pressure-derived features more prominence, or testing pressure trends/rates of change as additional features).